



1. Executive Summary & Core Purpose

PRONOVA is a modern, responsive, and minimalist project and checklist management platform engineered with **Next.js 16 (App Router)** and **React 19**. It is designed to provide high-speed task tracking and team collaboration by combining an instant, hydration-safe client state model with serverless backend capabilities.

KEY CAPABILITIES

- **Project Management:** High-level project scoping with lifecycle tracking (Planning, In Progress, Completed) and hardness priorities.
- **Checklists & Tasks:** Granular standalone checklist items or project-linked subtasks with live status toggling.
- **Timeline Alerts:** Categorized reminders (Deadlines, Meetings, Tasks, General) with automatic date grouping.

ARCHITECTURE HIGHLIGHTS

- **Zero-Latency UI:** Zustand 5.0 client-side state with automatic browser persistence (`localStorage`).
- **Identity Security:** Managed authentication and session protection powered by Clerk Edge middleware.
- **Transactional Email:** Multi-tiered outbound invite engine supporting Gmail SMTP and Resend API.

2. Complete Technology Stack

Layer / Domain	Technologies & Libraries	Role & Implementation Details
Frontend Framework	Next.js 16.2.4 & React 19.2.4	App Router architecture, TypeScript, Fast Turbopack bundler, and Server/Client Component division.
Styling & Design System	Tailwind CSS v4, Radix Base UI, Lucide Icons	Utility-first styling with HSL CSS variables, custom dark/light theme engine via <code>next-themes</code> , and accessible primitives.
Client State	Zustand 5.0.12 (with <code>persist</code>)	Store-based reactive state with automatic synchronization to browser <code>localStorage</code> .
Authentication	Clerk Next.js 7.2.5	Edge middleware route protection, secure session tokens, modal profile manager, and user attributes.
Database & ORM	Prisma 7.8.0, Neon PostgreSQL	Relational database models for Workspaces, Members, Projects, and Tasks with connection pooling.
Email Engine	Nodemailer 8.0.7 & Resend 6.12.3	Transactional workspace invitation delivery supporting SMTP direct relay and API fallback.

3. Project Directory & Component Breakdown

File / Directory Path	Responsibility & Key Operations
src/middleware.ts	Clerk Edge Middleware. Intercepts all traffic and protects non-public routes (except /sign-in and /sign-up).
src/app/layout.tsx	Global app shell wrapping all pages in ClerkProvider, ThemeProvider, Sidebar, and Topbar.
src/app/dashboard/page.tsx	Workspace command center: live KPI cards, project summary list, dynamic activity feed, and overdue task alerts.
src/app/projects/page.tsx	Project catalog with multi-dimensional filtering (status, priority, search) and creation/detail modals.
src/app/tasks/page.tsx	Personal checklist board with completion tracking, priority badges, and project filter bindings.
src/app/reminders/page.tsx	Smart alerts manager with dynamic grouping (<i>Today, Tomorrow, Upcoming, Past Due</i>).
src/app/team/page.tsx	Roster table displaying member status (<i>Active / Pending</i>), role management, and invitation dispatch.
src/app/settings/page.tsx	User settings: Light/Dark theme switch, workspace naming/slug configuration, and notification toggles.
src/app/api/team/invite/route.ts	Backend Node.js serverless route for generating and dispatching invitation emails via Nodemailer or Resend.
src/lib/store/*	Isolated Zustand stores (project-store, task-store, reminder-store, team-store, search-store).
prisma/schema.prisma	Relational database schema containing Workspace, Member, Project, Task models and enums.

4. Relational Data Architecture (Prisma & Neon PostgreSQL)

The project defines a normalized relational schema configured for **Neon Serverless PostgreSQL**. The schema captures workspace hierarchy, membership permissions, and nested task assignments:

```
datasource db {
  provider = "postgresql"
}

model Workspace {
  id          String    @id @default(cuid())
  name        String
  slug        String    @unique
  imageUrl    String?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
  members     Member[]
  projects    Project[]
}

model Member {
  id          String    @id @default(cuid())
  userId      String    // Clerk User ID reference
  role        Role      @default(MEMBER) // ADMIN | MEMBER
  workspaceId String
  workspace   Workspace @relation(fields: [workspaceId], references: [id], onDelete: Cascade)
  assignedTasks Task[]
  @@unique([userId, workspaceId])
}

model Project {
  id          String    @id @default(cuid())
  name        String
  description  String?
  status      ProjectStatus @default(PLANNING) // PLANNING | IN_PROGRESS | COMPLETED
  priority     Priority     @default(MEDIUM)   // LOW | MEDIUM | HIGH
  startDate   DateTime?
  endDate     DateTime?
  workspaceId String
  workspace   Workspace   @relation(fields: [workspaceId], references: [id], onDelete: Cascade)
  tasks       Task[]
}

model Task {
  id          String    @id @default(cuid())
  title       String
  description  String?
  status      TaskStatus @default(TODO) // TODO | IN_PROGRESS | DONE | BACKLOG
  priority     Priority   @default(MEDIUM)
  dueDate     DateTime?
  projectId   String
  project     Project    @relation(fields: [projectId], references: [id], onDelete: Cascade)
  assigneeId  String?
  assignee    Member?    @relation(fields: [assigneeId], references: [id])
}
```

5. Critical Diagnostic Findings (Why Emails Fail to Send)

1. Google App Password Spacing Issue in .env.local:

The SMTP_PASSWORD in .env.local is set to "mccv vtod rzet acda". While Google formats App Passwords with four 4-letter groups separated by spaces for human readability, Gmail SMTP authentication strictly expects the raw 16-character string without whitespace (mccvvtodrzetacda). Passing spaces causes Gmail SMTP to immediately reject authentication with error 535-5.7.8 Username and Password not accepted.

2. SMTP Configuration & Connection Port:

route.ts creates a raw transport using host smtp.gmail.com and port 465 (SSL). On many local Windows development machines and cloud networks, direct SSL connections on port 465 experience firewall drops or handshake timeouts. Configuring Nodemailer with service: 'gmail' or port 587 (STARTTLS) provides vastly higher connection reliability.

3. Silent Error Masking in UI:

In src/components/team/invite-member-modal.tsx and manage-member-modal.tsx, when the fetch('/api/team/invite') request fails, the component only logs to console.error and immediately closes the modal while adding the member to the local table as "Pending". The user sees no error alert, giving the false impression that the invitation succeeded.

4. Resend Sandbox Mode Restriction:

If SMTP credentials are bypassed, the system falls back to Resend API using onboarding@resend.dev. Resend's free tier restricts sending exclusively to the email address registered to that Resend account. Inviting any external email address results in a Sandbox restriction rejection.